

Testing & Code Quality Audit

TradeVault — Staff Engineering Review

Analyse : tests, TypeScript, qualite code, maintenabilite, tooling

Juillet 2026

- | | |
|-----------------|-------------------|
| 1. Tests | 4. Maintenabilite |
| 2. TypeScript | 5. Tooling |
| 3. Code Quality | |

1. Tests

Analyse des tests existants, couverture, tests critiques manquants, E2E.

22

Tests (22/100)

T01 CRITICAL (tous)

P0 | tests/ (tous)

[Probleme] 79 tests unitaires seulement pour ~25K lignes de code. Aucun test dans src/. Couverture estimee < 3%. Pas de tests pour: AuthContext, TradeModal (989 lignes), App.tsx (495 lignes), composants UI, stores, backend functions, webhooks, crons.

[Risque] Regression garantie lors de toute modification. Impossible de refactorer sans risquer de casser des fonctionnalites critiques. Chaque deploy est un pari.

[Solution] Implementer une strategie de tests par couches: unitaire (modules purs), integration (server functions), E2E (parcours utilisateur). Cibler 40% de couverture avant production.

T02 HIGH package.json

P1 | package.json

[Probleme] Pas de script 'test' dans package.json. Les developpeurs doivent connaitre 'bun test' pour lancer les tests. Aucun moyen standardise.

[Risque] Les tests ne sont jamais executes. Pas de CI, pas de pre-commit. Les 79 tests existants sont du code mort qui n'est jamais verifie.

[Solution] Ajouter 'test': 'bun test', 'test:watch': 'bun test --watch', 'test:coverage': 'bun test --coverage' dans package.json.

T03 HIGH (tous)

P1 | tests/ (tous)

[Probleme] Aucun test d'integration. Les server functions (billing, coach, AI, reports, push) n'ont aucun test. Les webhooks Stripe/Coinbase, le middleware auth, les crons ne sont pas testes.

[Risque] Les parties les plus critiques du systeme (paiements, auth, IA, rapports) n'ont aucune protection contre les regressions.

[Solution] Ajouter des tests d'integration pour chaque server function. Mocker Supabase et les providers externes. Tester les webhooks avec des payloads reels.

T04 HIGH (tous)

P1 | tests/ (tous)

[Probleme] Aucun E2E testing. Pas de Playwright/Cypress. Les parcours utilisateur complets (signup -> onboarding -> trade -> analytics -> IA) ne sont jamais testes automatiquement.

[Risque] Les workflows critiques peuvent etre casses par des changements frontend. Pas de confiance dans les deploiements.

[Solution] Mettre en place Playwright avec 5 tests E2E critiques: auth complet, CRUD trade, navigation pages, AI coach, settings.

T05 MEDIUM tsconfig.json

P2 | tsconfig.json

[Probleme] Le repertoire tests/ n'est pas inclus dans tsconfig.json. Les fichiers de test ne sont pas verifies par TypeScript. Les erreurs de type dans les tests passent inaperçues.

[Risque] Les tests peuvent contenir des erreurs de type silencieuses. Les mocks et fixtures peuvent diverger des types reels.

[Solution] Ajouter 'tests/**/*.*ts' dans le include de tsconfig.json. Creer un tsconfig.tests.json separe si necessaire.

T06 MEDIUM (tous)

P2 | tests/ (tous)

[Probleme] Pas de fixtures ou test helpers partages. Chaque fichier de test definit ses propres factories inline (mkTrade, trade(), fakeProvider(), voice()). Duplication de code de test significative.

[Risque] Fragilite: si le type Trade change, 10 factories inline doivent etre mises a jour individuellement. Risque d'oubli.

[Solution] Creer des factories centralisees (buildTrade, buildStats, etc.) dans un fichier tests/helpers.ts ou tests/factories.ts.

T07 MEDIUM (tous)

P2 | tests/ (tous)

[Probleme] Aucun mock pour Supabase, providers IA, ou API externes. Les tests existants testent uniquement des fonctions pures (computeStats, behaviorSignals, edgeScore, fallbackCoach).

[Risque] Aucun test ne verifie le comportement avec des donnees reelles ou des erreurs externes. Les cas de echec (Supabase down, provider IA qui plante) ne sont pas testes.

[Solution] Ajouter des mocks Supabase pour tester les stores et server functions. Tester les cas d'erreur: timeout, 429, invalid data.

2. TypeScript

Analyse du typage : any, assertions, non-null, securite.

62

TypeScript (62/100)

TS01 CRITICAL billing.server.ts

P0 | src/backend/billing.server.ts

[Probleme] 3 fichiers backend desactivent completement la regle no-explicit-any en tete de fichier. billing.server.ts, lifecycle-emails.server.ts, monthly-reports.server.ts utilisent des parametres typés any et des appels Supabase non types.

[Risque] Les fonctions de paiement, emails et rapports contournent entierement le systeme de types. Toute erreur de type dans ces fichiers devient une erreur runtime silencieuse.

[Solution] Remplacer les any par des types concrets (Zod schemas + inference). Supprimer les eslint-disable file-level. Typer correctement les appels Supabase.

TS02 HIGH goalPlan.ts

P1 | src/app/utills/goalPlan.ts

[Probleme] 10 occurrences de 'as unknown as X' a travers le codebase. Cette double assertion contourne completement la verification de type. Utilisees pour les colonnes JSON Supabase (trading_plan, goal_plans, rules).

[Risque] Aucune securite de type sur les colonnes JSON. Une erreur de structure dans la DB devient une erreur runtime silencieuse (undefined is not a function).

[Solution] Remplacer par des Zod schemas avec .parse(). Ajouter des parseurs types pour chaque colonne JSON.

TS03 HIGH usePushNotifications.ts

P1 | src/app/hooks/usePushNotifications.ts

[Probleme] 4 occurrences de '(supabase as any).from(...)' qui bypassent le typage Supabase. Meme pattern dans push.functions.ts et reports.functions.ts.

[Risque] Les requetes DB dans ces fichiers critiques (notifications push, rapports) ne sont pas typees. Une faute de frappe dans le nom de table ou de colonne passe en production.

[Solution] Utiliser le client Supabase type (Database type parameter). Si le typing est insuffisant, creer des fonctions wrapper typees par table.

TS04 HIGH Checklist.tsx

P1 | src/app/pages/Checklist.tsx

[Probleme] 25+ occurrences de non-null assertions (!) dans le code audio de Checklist.tsx. a.ctx!, a.master!, a.ctx!.currentTime, a.ctx!.sampleRate. Aucun guard runtime si le AudioContext n'est pas initialise.

[Risque] Crashes potentiels sur les navigateurs qui bloquent l'audio (iOS, certains Chromes). L'ecran noir avec erreur JS non capturee.

[Solution] Remplacer ! par des guards optionnels (?. ou if). Ajouter un wrapper AudioEngine qui encapsule la gestion d'etat.

TS05 MEDIUM MobileNav.tsx:26

P2 | src/app/components/MobileNav.tsx:26

[Probleme] NAV_ITEMS.find(...)! — non-null assertion sans fallback. Si l'item n'est pas trouve (par exemple navigation future), l'app crash.

[Risque] Crash silencieux de la navigation mobile. Impossible de naviguer.

[Solution] Remplacer par find(...) ?? fallback avec gestion d'erreur.

TS06 MEDIUM (tous)

P2 | src/modules/ (tous)

[Probleme] 8 fichiers modules/ importent depuis @/app/types (Trade, TradingRule, generateld). Le sens des dependances est inverse: modules/ ne devrait jamais importer depuis app/.

[Risque] Les modules ne peuvent pas etre extraits, testes ou reutilises independamment. Couplage fort entre le coeur metier et l'UI.

[Solution] Deplacer les types partages (Trade, TradingRule) dans modules/. Deplacer generateld dans shared/. app/ importe depuis modules/, jamais l'inverse.

TS07 MEDIUM tradingRules.ts

P2 | src/app/utis/tradingRules.ts

[Probleme] Record<string, unknown> utilise pour les colonnes JSON sans validation runtime. Les memes patterns dans goalPlan.ts, tradingPlan.ts.

[Risque] Donnees JSON malformees en base = crash silencieux a la lecture. Pas de migration possible sans risque de casser le parsing.

[Solution] Ajouter Zod.parse() a chaque point de lecture des colonnes JSON. Logger les echecs de parsing pour detecter les corruptions.

3. Code Quality

Composants fragiles, fonctions trop grandes, duplication, mauvaises pratiques.

48

Code Quality (48/100)

CQ01 CRITICAL Checklist.tsx

P0 | src/app/pages/Checklist.tsx

[Probleme] 2030 lignes dans un seul composant. Melange: audio engine (200 lignes), voice synthesis, wizard integration, animations, persistence, 12 useEffect, 7 useMemo, 20+ useCallback, 88 divs. Veritable 'god component'.

[Risque] Impossible de tester, deboguer, ou refactorer sans risque. Toute modification a un impact non localise. Point de friction majeur.

[Solution] Decouper: extraire AudioEngine dans un hook useAudioEngine(), extraire les sous-composants (Ed, Item, SessionTimer, ConfigPanel), extraire la persistence dans useChecklistPersistence().

CQ02 HIGH App.tsx

P1 | src/app/App.tsx

[Probleme] 495 lignes orchestrant 7 etats, 10 handlers, 6 effets. handleSave() enchainé: cache optimistic -> Supabase -> rollback -> balance -> Automation Engine. handleDelete identique. Provider nesting 6 niveaux.

[Risque] Composant le plus critique du systeme et le plus fragile. Une modification dans handleSave peut casser la creation/sauvegarde de tous les trades.

[Solution] Decouper: AppProviders.tsx pour le provider shell, useTradeCrud() pour les handlers CRUD, useAppEffects() pour les effets globaux.

CQ03 HIGH Landing.tsx

P1 | src/app/pages/Landing.tsx

[Probleme] 1412 lignes avec 6 sous-composants definis inline, 5 blocs de donnees statiques, 3 hooks inline. 85% presentation mais impossible de tester ou reutiliser les sous-composants individuellement.

[Risque] Maintenance lourde. Le marketing voudra modifier la landing page frequemment. Chaque changement risque de casser les animations ou le comportement existant.

[Solution] Extraire les sous-composants dans Landing/ directory. Extraire les constantes de donnees dans landingData.ts. Extraire les hooks dans un fichier separe.

CQ04 HIGH TradeModal.tsx

P1 | src/app/components/TradeModal.tsx

[Probleme] 989 lignes. 920 lignes dans la fonction principale. Melange: form state, image compression, paste events, auto-save draft, position calculator. 640 lignes de JSX retourne.

[Risque] Composant central de la creation de trade. 3 return distincts dans le JSX. Toute modification du formulaire impacte 1000 lignes.

[Solution] Decouper: TradeForm (fields), ScreenshotUploader, PositionCalculator, useTradeDraft() pour le draft auto-save.

CQ05 MEDIUM TradeDetailModal.tsx

P2 | src/app/components/TradeDetailModal.tsx

[Probleme] 495 lignes. Pattern de duplication avec TradeModal (screenshots, mistakes, confluences rendering). Memes classes Tailwind recopiees.

[Risque] Les changements d'affichage des screenshots/mistakes doivent etre faits dans 2 fichiers. Risque de divergence.

[Solution] Extraire SharedTradeDisplay (screenshots, mistakes chips, confluences chips, notes) reutilise par les deux modals.

CQ06 MEDIUM Analytics.tsx

P2 | src/app/pages/Analytics.tsx

[Probleme] 1102 lignes. Melange de computation de stats, rendu de 10+ charts Recharts, etats de chargement/vide. Les fonctions formatter utilisent ':' any' (5 occurrences) a cause du typing Recharts.

[Risque] Difficile de modifier un chart specifique sans tout parcourir. Les formatters any propagent un mauvais exemple.

[Solution] Extraire chaque section de chart dans un composant dedie (EquityCurveSection, DistributionSection, StrategySection, etc.).

CQ07 LOW Skeleton.tsx

P3 | src/app/components/Skeleton.tsx

[Probleme] 601 lignes avec 14 composants exportes. Beaucoup de duplication de classes Tailwind entre les variantes de skeletons.

[Risque] Faible impact mais signe que l'abstraction pourrait etre amelioree. Les skeletons changent rarement.

[Solution] Creer un composant Skeleton de base avec props shape et width/height. Reduire a 3-4 variantes.

4. Maintainabilite

Facilite de modification, risques futurs, zones dangereuses.

44

Maintenabilite (44/100)

M01 CRITICAL (8 fichiers)

P0 | src/modules/ (8 fichiers)

[Probleme] 8 fichiers modules/ importent depuis @/app/types et @/app/utlis. modules/discipline/engine.ts importe checkTradeAgainstRules depuis app/utlis/tradingRules qui elle-meme appelle Supabase. La couche discipline (pure engine) a une dependance runtime vers la couche UI/DB.

[Risque] La regle d'architecture fondamentale (modules/ n'importe jamais app/) est violee. Impossible d'extraire ou tester les modules independamment. Le couplage va s'aggraver avec l'ajout de nouvelles fonctionnalites.

[Solution] Deplacer Trade dans modules/trading/types.ts. Deplacer TradingRule et checkTradeAgainstRules dans modules/discipline/. Deplacer generateId dans shared/id.ts. Inverser les dependances.

M02 HIGH AuthContext.tsx

P1 | src/app/contexts/AuthContext.tsx

[Probleme] Le contexte auth appelle directement Supabase (signInWithPassword, signUp, signInWithOAuth, resetPasswordForEmail, signOut, getSession, onAuthStateChange, functions.invoke). Couplage fort entre React et Supabase SDK.

[Risque] Changer de provider auth (Clerk, custom backend, Auth0) necessite de reecrire entierement AuthContext. Migration impossible sans tout casser.

[Solution] Creer une couche d'abstraction auth (src/integrations/auth/interface.ts) avec implementation Supabase. AuthContext n'importe que l'interface.

M03 HIGH (10+ fichiers)

P1 | src/app/App.tsx + store/ (10+ fichiers)

[Probleme] Les stores (trades.ts, profile.ts, accounts.ts, reports.ts, missed.ts) appellent Supabase directement. Pas de couche d'abstraction entre le stockage et la logique applicative.

[Risque] Migration Supabase -> autre base impossible. Changer de schema de table necessite de modifier chaque store individuellement.

[Solution] Creer src/integrations/supabase/queries/ avec un fichier par domaine (trades.ts, profile.ts, accounts.ts). Les stores n'importent que les queries.

M04 HIGH Checklist.tsx (audio)

P1 | src/app/pages/Checklist.tsx (audio)

[Probleme] 200 lignes de code Web Audio API inline dans un composant React. AudioContext, oscillateurs, gains, enveloppes, timing. Aucun test, aucune abstraction, aucune gestion d'erreur pour les navigateurs qui bloquent l'audio.

[Risque] Bug audio difficile a reproduire et deboguer. Blocage sur certains navigateurs mobiles. Le composant de 2030 lignes devient encore plus dur a maintenir.

[Solution] Extraire dans un module src/modules/audio/ avec interface propre. Le composant React n'appelle qu'un hook useAudioEngine().

M05 MEDIUM (8 fichiers)

P2 | src/app/ (8 fichiers)

[Probleme] 8 fichiers utilisent window.dispatchEvent(CustomEvent) pour la communication inter-composants (tv-rules-updated, tv:ask-coach, tv:trustpilot-nudge). Aucun typage, aucune traceabilite.

[Risque] Les listeners CustomEvent sont fragiles: pas de typage, pas de verification a la compilation, pas de garbage collection si le composant est demonte sans cleanup.

[Solution] Remplacer par le bus d'evenements existant dans modules/events/bus.ts qui a deja l'infrastructure (emit, on, off, once, typage).

M06	MEDIUM	(tous)
P2 src/app/hooks/ (tous)		
[Probleme] Les hooks (usePushNotifications, useSubscription, useRealtimeProfile) appellent Supabase directement. Pas de separation entre la logique de hook et l'acces aux donnees.		
[Risque] Les hooks ne sont pas testables sans mocker Supabase. La logique d'abonnement et de notification est melangee avec le stockage.		
[Solution] Faire appel aux fonctions store/ depuis les hooks, pas a Supabase directement. Les stores deviennent la seule couche d'acces aux donnees.		

5. Tooling

ESLint, formatage, CI/CD, hooks pre-commit.

36

Tooling (36/100)

TL01 CRITICAL

P0 | .github/workflows/

[Probleme] Aucun CI/CD pipeline. Pas de GitHub Actions, pas de validation automatique. Le lint et les tests ne sont jamais executes avant deploiement. Vercel auto-deploie chaque push sans verification.

[Risque] Du code qui casse le lint, les tests, ou le typecheck est deployee en production. Regression garantie. Pas de gate de qualite.

[Solution] Creer .github/workflows/ci.yml avec: bun install -> bun run lint -> bun run typecheck -> bun run test. N'autoriser le deploiement Vercel qu'apres passage du CI.

TL02 CRITICAL package.json

P0 | package.json

[Probleme] Pas de script 'typecheck'. Le compilateur TypeScript (tsc --noEmit) n'est jamais invoque. Les erreurs de type ne sont detectees que par l'IDE du developpeur.

[Risque] Les erreurs de type passent en production. Le refactoring est risqué car aucune verification garantit que le code compile.

[Solution] Ajouter 'typecheck': 'tsc --noEmit' dans package.json. Executer dans le pipeline CI. Activer noUnusedLocals et noUnusedParameters dans tsconfig.

TL03 HIGH package.json

P1 | package.json

[Probleme] Pas de hooks pre-commit. Aucun husky/lint-staged. Les commits peuvent contenir du code qui ne passe pas le lint ou les tests.

[Risque] Le lint et les tests sont contournables. La qualite du code dans le depot se degrade progressivement.

[Solution] Ajouter husky + lint-staged. Executer eslint --fix et prettier --write sur les fichiers modifies avant chaque commit.

TL04 HIGH tsconfig.json

P1 | tsconfig.json

[Probleme] noUnusedLocals: false, noUnusedParameters: false. Les variables et parametres inutilises ne sont pas erreurs par TypeScript (seulement warns ESLint). exactOptionalPropertyTypes et noUncheckedIndexedAccess ne sont pas actives.

[Risque] Code mort qui s'accumule. Acces aux tableaux non verifies (crashes potentiels). Proprietes optionnelles permissives.

[Solution] Activer noUnusedLocals, noUnusedParameters, noUncheckedIndexedAccess, exactOptionalPropertyTypes. Corriger les erreurs resultantes.

TL05 MEDIUM eslint.config.js

P2 | eslint.config.js

[Probleme] Pas de regle no-floating-promises. Les Promesses non gerees (sans await, sans .catch()) ne sont pas detectees. Les erreurs async silencieuses passent inaperçues.

[Risque] Les erreurs dans les appels async (par exemple fire-and-forget) sont avalees silencieusement. Debogage impossible.

[Solution] Ajouter @typescript-eslint/no-floating-promises: error. Corriger les quelques occurrences dans le code.

TL06 LOW eslint.config.js

P3 | eslint.config.js

[Probleme] Pas de regles d'ordre d'import, pas de .editorconfig, pas de configuration VS Code partagee (extensions.json).

[Risque] Inconsistance de style entre developpeurs. Configuration manuelle de l'environnement a chaque nouveau membre.

[Solution] Ajouter eslint-plugin-import avec regles d'ordre. Ajouter .editorconfig. Ajouter .vscode/extensions.json avec plugins recommandes.

Scores par Domaine

22	Tests (22/100)
62	TypeScript (62/100)
48	Code Quality (48/100)
44	Maintenabilite (44/100)
36	Tooling (36/100)

Score Qualite Logiciel : 42/100

Ce score reflète la qualité technique du logiciel du point de vue d'un Staff Engineer. Il prend en compte: la couverture de tests, la rigueur TypeScript, la qualité du code, la maintenabilité et l'outillage. Un score < 50 indique un besoin urgent d'investissement dans la qualité avant de pouvoir évoluer sans créer de régressions.

Strategie de Tests Recommandee

Plan de mise en oeuvre progressif pour atteindre une couverture viable avant production.

Phase 1 — Foundation (Sprint 1, J1-J5)

Objectif: rendre les tests executables et bloquer le code non conforme.

- Ajouter les scripts npm: test, test:watch, test:coverage, typecheck

Phase 2 — Tests critiques (Sprint 2, J6-J15)

Objectif: couvrir les chemins critiques de maniere exhaustive.

- Auth: login, signup, OAuth, password reset, session handling, delete account

Phase 3 — E2E (Sprint 3, J16-J25)

Objectif: valider les parcours utilisateur complets.

- Playwright: auth complet (inscription -> login -> deconnexion)

Phase 4 — Robuste (Sprint 4, J26-J40)

Objectif: atteindre 50%+ de couverture et automatiser la qualite.

- Tests de performance: cron N+1 avec 1000 utilisateurs simules

Cibles de couverture par couche :

- Modules purs (trading analysis, discipline, events, voice): 90%+ coverage